Latent Causal TAMP

Implementation and experimental guide v1.7 | 2D, 3D and G3.5 migration

16 October 2026

Start here: the new implementation contract

This guide implements Methodology Specification v1.5. It replaces Guide v1.6 for future primary work while preserving the archived G0-G3 frozen-motion controls. It is a plan and acceptance contract, not a claim that the code described below already exists.

User's agreed interface: initial scene plus robot state, object information and intended action/goal in; obstacle IDs and relocation destinations out. No trajectory or swept-volume input to the learned model. External IK and motion planning execute those relocations. The target planner evaluates the intended action after the selected repair, without influencing primary proposal selection.

Protocol boundary

Component	May access	Must not do
Proposal model/sampler	Initial learner observation and hypothetical endpoints	Read reference paths, target outcomes or later images
Static screen	Candidate endpoint geometry	Claim target-path existence
Relocation executor	Initial/achieved states, candidate assignment, IK and collision backend	Invoke target evaluator to rank candidates
Target evaluator	Final achieved state, action/goal and constraints	Send its result back to change the committed repair
Offline teacher	Full simulator and planner evidence	Leak privileged fields into learner tensors

Primary protocol string: `INITIAL_SCENE_ONE_SHOT_V1`. Archived protocol string: `FROZEN_CONTROL`. A future proposal-refresh/retry method requires a different protocol string and separate results.

What to do next

Run **G3.5 only**, using `G3_5_initial_scene_repair_migration.md`. It adds the new objective boundary, cases, evidence schema and regression tests. Do not begin G4 scaling or G5 training. The first G3.5 candidates are manually specified or deterministic; no learning performance is claimed.

The supplied G3 report records a tested full 3R body with 48 grasp candidates, exact fixed-motion matching, executable repair witnesses, and 74 passing tests. These are reported historical results, not newly audited results. Read the actual repository before changing code. Its tested profile and asset/configuration hashes override provisional numeric examples in older guides.

Contents

1. Minimal file ownership; 2. configuration; 3. data records; 4. G3.5; 5. case analysis; 6. offline labels and G4; 7. model and G5; 8. evaluation and G6; 9. 3D and G7-G9; 10. long horizon and G10; 11. visual/evidence contract; 12. worked examples and release checks.

1. Files and ownership: keep the existing project small

Inspect the current tree first. Reuse existing modules, functions and CLI conventions. Do not force the repository back into the provisional file list of v1.6. The G3 report names `skill.py`, `config.py`, `figures.py`, `repair_figures.py` and `assignment_figures.py`; retain those established responsibilities.

File or group	Responsibility after migration
`scene.py`, `config.py`, `fixtures.json`	State, stable IDs, physical candidates, structured action/goal and strict configuration
`geometry.py`	Static support/pair checks and primitive geometric certificates
`motion.py`, `skill.py`	Existing FK/IK, full-body checks and complete skill planning
`interventions.py`	Offline matched edits and changed-field audit
`solve.py`	Keep frozen A0; add deterministic migration proposals, later new Gibbs scoring
`verify.py`	Relocation order/witness verification and achieved-state continuity; protocol branching
`evaluate.py`	Post-commit target evaluation and independent baseline/control runs; create at G3.5 only if needed
Existing figure modules	Scene-derived comparisons and replay; no duplicate visualization stack
`data.py`	Learner allowlist, label masks and root splits; create only when needed
`model.py`, `learn.py`	New local-plus-assignment model and offline training at G5
`run.py`	Thin dispatcher and I/O; relocate stage bodies into their owning modules
`tests/`, `runs/`, `assets/`	Tests, flat run records and pinned 3D assets

No generic framework, registry, database or parallel 3D source hierarchy. Prefer plain dataclasses, JSON and NPZ. Ordinary modules should remain roughly 120-280 logical lines. If an actual module exceeds about 300 lines, split one clear responsibility and explain the reason. Do not create empty future-stage files.

The G3 report has `run.py` at 318 logical lines. A narrow dispatcher split is allowed during G3.5 if required for the new command; avoid unrelated rewrites and preserve archived CLI/replay behaviour.

Recommended new command shape is `python run.py case-analysis --config config.toml --out runs/g3_5_initial_scene`. Adapt the verb if the actual CLI already has an equivalent; document the exact command used. Preserve `solve` and `replay` for frozen controls.

One flat output directory per stage/run: `config.json`, `result.json`, `cases.json`, `metrics.csv`, `manifest.json`, `overview.png`, `arrays.npz`, case PNG/GIF files and optional training checkpoints. Do not create one directory per frame or duplicate all original archives. Use references to immutable legacy evidence when appropriate.

Dependency direction: proposal code cannot import or call the target evaluator. Inject the target callback only into the post-commit orchestration layer. The offline teacher can explicitly call both relocation and target components in its own labeling path.

2. Configuration registry and preserved physical settings

Use Python `tomllib`, reject unknown keys, serialize the resolved configuration and hash it. Keep separate seeds for root generation, interventions, rendering, model initialization, sampler, relocation planning and target evaluation. Target evaluator seeds are never proposal inputs.

Preserve actual tested robot settings. The source report gives a robot profile hash beginning `2d7986761356f926`. Compare its full recorded hash with the repository; if it differs, explain why rather than forcing a match. Preserve the tested home, gripper, layer/contact rules, 48-candidate grasp family and conservative checking policy. Old example `q_home=[0,0,0]` is not an instruction to reset the robot.

Retained benchmark values: 8×8 resting grid; pitch 0.08 m; obstacle radius 0.024 m; clearance 0.005 m applied once; geometry float64; network float32. Dense 2D target is 30-40 objects, with 35 as the initial challenge setting. Sparse development cases precede density. Record non-grid/heterogeneous exceptions explicitly; matching exactness does not extend automatically.

New settings (proposed defaults; validate at G3.5)

```toml
[protocol]
name = "INITIAL_SCENE_ONE_SHOT_V1"
proposal_observation = "INITIAL_ONLY"
target_feedback_to_selection = false
target_calls_between_relocations = 0
target_reference_required = false
execution_observation = "VALIDATE_OR_ABORT"
move_once = true

[proposal]
source = "DETERMINISTIC_CASE_CANDIDATES" # G3.5 only
budget_cap = 6
max_candidates_to_verify = 12
include_all_stay = true

[target_evaluation]
when = "AFTER_REPAIR_COMMIT_AND_COMPLETION"
start_state = "ACHIEVED"
scope = "COMPLETE_TARGET_SKILL"
max_queries_per_primary_episode = 1
```

One target query means one evaluator invocation with a predeclared finite internal search schedule; it does not mean one IK sample. Pin the underlying planner node/evaluation/time caps to the tested configuration. Do not copy the old 30 s example if the current backend uses another validated cap. Report all internal attempts and computation.

G3 reported search defaults: k-best 12, at most 6 non-dominated orders, 40 relocation plans and 2 failures per skill. These describe the frozen search; they are starting caps for explicit migration testing, not automatically fair settings for every new baseline. Keep per-case wall-time limits to prevent dense probes consuming the stage. Record any changes and resulting witness differences.

Protocol validation must reject contradictory settings, e.g. `INITIAL_ONLY` with proposal refresh, nonzero between-move target calls, or a required frozen path in the new mode.

3. Data records, hashes and API boundaries

Use small records or equivalent existing dataclasses. Avoid replacing working state machinery for cosmetic schema consistency.

Record	Required fields
Query	protocol, initial scene, robot state, action, goal, constraints, candidates, observation hash, action-context hash
Proposal	source-observation hash, endpoint assignment, moved IDs, score, rank, seed, budget; no target outcome
Commitment	selected proposal hash, assignment, chosen order, relocation witness hash, commit event index
RepairState	achieved geometry, joints, aperture, attachment, moved history, episode ID
Evidence	property tested, status, proof/witness scope, reason, bounds, relevant state hash and checker settings
EpisodeResult	component statuses, target call trace, costs, achieved goal, success/abort, no hidden retries

`action_context_hash` hashes action parameters, goal, constraints and robot profile; it is not a trajectory hash. Trajectory hashes remain for execution replay and optional archived controls. A primary query must load successfully when all reference trajectory/envelope fields are absent.

Assignment evidence

Store `static_status`, `assignment_feasibility_status`, `relocation_attempt_status`, `target_attempt_status`, `simulation_status`, `hardware_status`, `certificate_scope`, `lower_bound`, `upper_bound`, `minimality_status`, and `unknown_reason` independently. A specific order rejection does not set assignment feasibility to zero. Target timeout after a legal repair is unresolved existentially and unsuccessful operationally within the budget.

The initial scene is immutable for proposal generation. The execution state is mutable and records achieved states. A simulated candidate check uses a clone, never silently changes the live state. For dynamic/hardware execution, command and achieved pose are distinct fields.

Event trace

Emit ordered events with protocol, episode ID, state hash, component name and call count: `PROPOSE`, `STATIC_CHECK`, `REPAIR_PLAN`, `COMMIT`, `RELOCATE`, `REOBSERVE`, `TARGET_EVALUATE`, `FINISH` or `ABORT`. Tests assert there is no `TARGET_EVALUATE` before commitment, no such event between relocations, and no new proposal/commitment after its result in the same primary episode.

R0/B0 initial-feasibility checks and offline teacher searches use a separate control/evidence namespace and cloned state. Run them independently of the primary proposal and never pass their results to it. Reporting initial feasibility is allowed; using that result to pick a primary repair is not.

Learner allowlist

The loader constructs tensors from explicit allowed keys, never by flattening a record. Poison all `oracle_only` values, proof labels, planner traces, root IDs and reference fields and confirm tensor equality. Remove reference fields altogether and confirm the primary query still loads. Reordering object slots must permute per-object outputs and leave assignment energy invariant when the assignment is permuted consistently.

4. G3.5: migration and interface case analysis

Objective: prove that the new problem definition can be expressed, evaluated and recorded without modifying the scientific meaning of G3. Do not train or generate a large corpus here.

Step A - inspect and preserve

Read repository instructions, reports 0001-0006, the G3 instruction file, current configs and relevant source/tests. Record current branch, commit, working tree and archive hashes. Preserve unrelated user edits. Establish a separate new run path. Do not rename, regenerate or overwrite old G0-G3 output files.

Step B - make the narrow code change

Add an explicit new protocol. Split endpoint geometry from frozen target clearance. Keep the latter reachable only for `FROZEN_CONTROL` or privileged offline comparison. Make the relocation verifier optionally end at the actual last retreat state; retain exact frozen-start connector behaviour in the legacy mode. Introduce a post-commit target entry point that plans the full target skill from that actual achieved state.

Use deterministic or manually supplied candidate assignments. The primary experiment must not pick one using target success. The post-commit evaluator records whether the chosen candidate worked; it cannot retry candidates based on that result. A separate offline multi-candidate case audit may label every candidate, clearly marked as privileged.

Step C - implement R0-R6

Use the case table on the next page. First find small valid 3R witnesses. Reuse archived D1-D3 and dense T07 as starting geometry when useful, but issue new protocol/root-query identifiers and new evidence. Never simply copy their blocker counts or minima into new rows.

Step D - test actual risks

Required tests: primary input works without a reference path; privileged fields cannot alter tensors; static-valid is not target-success; target callback ordering; achieved-state target start; history/move-once enforcement; unknown masking; no target-feedback fallback; frozen legacy connector/hash behaviour; candidate-clone isolation. Exercise a real complete target plan, not only a mocked callback. Spies can test the architectural boundary in addition to actual planning witnesses.

Step E - report and stop

Write `0007_G3_5_completion.md` with actual cases, changed files and line counts, config/profile hashes, test commands, visual paths, failure/unknown reasons and the next-stage readiness matrix. State that candidates were deterministic and no learned improvement was measured. Preserve old reports. Follow existing repository synchronization rules and authorization; report sync state honestly.

Stop before G4. Required missing cases or evidence block the relevant readiness item. Do not erase unresolved attempts to produce a clean-looking report.

5. G3.5 case matrix and required visual outputs

Case	Implementation task	Acceptance evidence
R0	Complete the target with zero relocations	Full target witness, empty history, minimum zero
R1	Find a scene where the old frozen path collides but another target path succeeds without relocation	Frozen collision evidence plus complete alternative zero-move witness from the same initial robot state
R2	Occupy a unique mandatory target goal with one obstacle, then move it to a legal destination	Checked goal-overlap necessity certificate plus one-move repair and target witness
R3	Test two different sufficient assignments from one initial scene	Two independent complete witnesses; teacher alternatives retained, no assumed minimum
R4	Compare two orders for the same assignment	Actual first violation/unknown for one order and a full witness for another, including final target action
R5	Test static-valid endpoints that do not yield a target witness	Separate static positive and target certified-negative/unknown; no false label conversion
R6	Exercise invalid/unknown and data leakage controls	Same-cell rejection, support rejection, finite-budget unknown, provenance tests

Do not promise R1 exists in a particular archived dense fixture. Search a modest set of small candidate scenes under the actual profile. If no witness is found, document the search and mark the case unavailable; do not assert alternative-path impossibility. Gate G4 on demonstrating the key semantic distinction in some eligible full-robot scene.

For R2, the required goal must be a fixed pose with no allowed alternative orientation/region that avoids overlap. The obstacle must remain stationary unless selected for repair; no incidental pushing is permitted. These premises make a lower bound of one sound. A complete one-move witness closes it. If the premises cannot be met, retain the witness but omit the optimality claim.

R3 need not involve two necessary movers. Two alternative one-move repairs or two different destinations suffice if both have complete witnesses. R4 may use G3 D3's assignment as a starting point, but verify the newly planned target after the successful order; old target replay alone does not validate a changed operational evaluator.

Required figures

1. `A_protocol_cases.png`: initial scenes, action/goal, predicted endpoints, move counts and three component statuses for R0-R6.
2. `B_r1_route_comparison.png`: same scene; failed frozen reference in one panel, actual new zero-move target witness in another. Label the former as a privileged diagnostic.
3. `C_r2_repair.gif`: initial state, real obstacle relocation, achieved state and newly planned target action. Include gripper phase and moved history.
4. `D_r3_alternatives.png`: both complete repair alternatives with their actual evidence and endpoints.
5. `E_r4_orders.png`: rejected/unresolved order and verified order, including final robot-state differences and target outcome.
6. `F_call_trace.png`: event timeline with commit and target-evaluation boundary visibly separated; target calls before/between moves must be zero.

For missing evidence use `NOT_RUN`, `UNRESOLVED` or `CONSTRUCTION_UNAVAILABLE` in the panel. Never animate a failed motion using invented interpolation. The clean RGB supplied to the learner is stored separately from figures with arrows, paths or labels.

6. G4: labels, proposal engine and compact dataset pilot

Begin only after G3.5 acceptance and next-stage authorization. Reuse the same geometry and full-robot backend. Add only the smallest `data.py`/solver changes needed for the allowlist, root splits, finite-pool labels and correct full-energy Gibbs updates.

G4A - independent evidence generation

Generate simple roots first, including no-repair successes, goal-occupancy cases, alternative repairs, static competition and order/access dependence. A frozen reference may help construct a root, but it is optional privileged provenance. Independently evaluate the new complete-assignment objective for every label used in task-head supervision.

For each candidate: check static factors; if statically valid, attempt legal relocation orders on clones; for each completed repair, evaluate the target from its final state. Save a positive once a complete witness is found. Save a negative only with a sound certificate, including static invalidity or properly scoped unique-goal obstruction. Search exhaustion is unknown. A teacher may search multiple candidates/orders because it is offline; its outcomes never enter primary inference.

Archive all labels with proof/witness hashes, exact state, budget, seeds and scope. Keep a histogram of negative sources and of unknown reasons. Explicitly distinguish a failed order from an excluded assignment.

G4B - sampler correctness

Implement the whole-assignment energy interface before fitting a neural network. Test with a small synthetic score over categorical assignments so exhaustive probabilities are available. Compare global-term conditional log probabilities and full energy differences; test budget compliance, all-stay reachability with soft factors, seed reproducibility and deduplication. This synthetic test validates sampler mathematics only, not robot feasibility.

Do not use the exact action oracle as an online scorer for a baseline labelled equal-information learned inference. An oracle-assisted sampler is a named privileged offline/control method. Deterministic geometry and uniform baselines remain available for development.

G4C - data readiness and scale

Audit 20 independent roots before a 300-root pilot (210 train, 45 validation, 45 test). Split roots before descendants. Freeze test identities and do not inspect their outcomes for architecture tuning. Expand toward 1,000 roots only if generation cost, witnesses and informative decided labels justify it. Report actual counts separately for 2D and later 3D.

Readiness requires multiple complete repair alternatives, zero-move successes, decided static factors, informative action-level negatives beyond trivial duplicate cells, and unresolved records kept separately. Set numerical coverage targets in the run plan before scaling; if the generator cannot meet them, report the limitation and revise the data plan before training.

J02 is redefined as a new-protocol teaching case, not relabelled in place. Verify each candidate assignment independently. One-move candidates may succeed via a different target route; they are not automatically negative because an old direct blocker remains. Preserve the original J02 under the frozen-control namespace.

Pass artifacts: split manifest, loader audit, label-source table, actual candidate/witness visualizations, per-root throughput and sampler validation. No loss curve is an acceptance artifact for this stage.

7. G5: minimal model, losses and training configuration

Create `model.py` and `learn.py` only after G4. Follow specification v1.5 Sections 5-7 exactly. The model learns static unary/pair compatibility plus a whole-assignment success score; no frozen-motion action-clear head remains in the primary architecture.

Starting architecture and settings

Item	Initial setting
Visual backbone	Frozen DINOv2 ViT-S/14 or explicitly documented available equivalent
Object features	Mask-pooled appearance, known shape, initial and candidate poses, support
Action context	Initial robot/gripper state, target, goal, workspace and skill constraints
Local heads	Shared two-layer width-128 MLPs; pair exchange symmetry
Assignment head	Shared width-128 token MLP, sum pooling with count/context, two-layer readout
Energy	Static unary + pair + weighted assignment success energy
Optimizer	AdamW; learning rate 10^{-3}, weight decay 10^{-4}
Initial loss weights	$\lambda_s = 1$, $\lambda_J = 1$, $\lambda_{task} = 1$
Pool temperature	$T_{train} = 1$
Finite pool	Target about 32 decided assignments where available; log actual size
Model seeds	Three independent seeds

These are proposed starts, not tested optimum hyperparameters. Pin actual software/device versions after a smoke test. Do not copy an obsolete Torch/CUDA pin from unrelated work. Save feature-backbone revision and preprocessing, normalization transforms, split/config/robot hashes and tensor-schema version in every checkpoint.

Train static factors and the assignment head with their decided BCE labels first; then add the finite-pool joint loss. Use the same positive meaning (complete repair-plus-target witness) in task BCE and joint loss. Unknown candidates are excluded from the joint denominator. All-negative/all-positive pools do not provide the intended mixed feasibility contrast; log and skip that term.

Validate the joint gradient sign by finite differences. Check that nonnegative softplus energies remain finite, pair symmetry holds, assignment score is permutation invariant, and global Gibbs conditionals match recomputed full energies. Recompute ghost geometry at each candidate pose. The source observation stays initial throughout sampling.

Choose task-score threshold τ on validation data using explicit success/move/coverage tradeoffs; freeze it before test. Do not present $\sigma(u)$ as calibrated physical probability unless calibration is separately measured. Show a reliability curve if probability language is used.

Required visualizations: actual initial scene, candidate assignments with model scores, oracle statuses and complete witness outcomes; learning curves separated by static/task/joint loss; success versus moved count and unknown coverage. Mark the stage's test set untouched until G6.

Pass: no leakage, correct losses/sampler, reproducible training/checkpoint reload and honest validation behaviour. If the model only learns easy static negatives, state that and do not claim action-feasibility learning.

8. G6: matched evaluation and claims gate

Implement the B0-B9/O1/F0 registry in specification v1.5. Preserve archived A0-A11 names for historical protocols. Minimum primary comparison is B0, B1, B2, B3 and B4. Add B5/B6/B8/B9 before claiming benefits of whole-assignment conditioning, Boltzmann sampling, interventions or joint loss respectively.

One fixed candidate domain, observation interface, relocation backend and target evaluator is used across matched methods. Static screening is shared. For each method freeze proposal count, move cap, repair-planner budget, target evaluation budget, seeds and selection policy before test. Record all failures and all eligible roots.

No target look-ahead in the primary method. Instrument target calls. A target-oracle-assisted search may be informative as O1, but it cannot be described as the same-information baseline. An initially feasible baseline result can classify the evaluation cohort only after the primary proposal has been independently fixed.

Required per-query columns

Protocol; root/query ID; initial observation and action hashes; domain policy; method/checkpoint; proposal seed; proposed assignment and moved count; rank and score; static status; commitment; relocation order and witness; achieved robot/scene state; target outcome; complete success; abort/unknown reason; call trace; time breakdown; proof scope and lower/upper bounds. Reference trajectory IDs belong only to optional diagnostics.

Metrics and denominators

Metric	Denominator / qualification
Overall complete success	Every eligible query, including abstention and unresolved execution
Relocation success	Every eligible query; empty repair is a completed relocation sequence
Target success after completed repair	Queries with completed repair, including all-stay
Unnecessary moves	Queries independently witnessed initially feasible
Repair success on initially infeasible	Only certified initially infeasible cases; separately show initially unresolved cohort
Minimum-move attainment	Only soundly certified action-level optima
Causal validity	Sufficient repair execution and explicitly scoped paired effects

For success versus move budget, run independently specified episodes under each cap, not incremental target-guided retries hidden inside one episode. Report median and tail latency; separate network setup, sampling, static screening, relocation planning/execution and final target planning. Cluster bootstrap over roots, not siblings.

Use matched appearance-versus-geometry comparison for identical cylinders. B8 intervention and observational datasets must match roots, label budget, family mix and decided-label coverage. For pair removal compare an inference lesion and a retrained counterpart before making representation claims. For B9 hold examples constant while changing only the joint objective.

Pass: every aggregate row is traceable to a recorded episode, no denominator excludes inconvenient unknowns, and all claims remain within measured evidence. If geometric baselines win, report that result and narrow the contribution.

9. G7-G9: transfer the same contract to 3D

G7 - assets, contact policy and sparse development

Pin full xArm7 and gripper assets, licenses/revisions, joint limits, mounting transform, TCP, validated home and tool-to-object transform. Use the existing module/backend structure. Do not replace the robot with a planar proxy for 3D claims.

Proposed cupboard profile: interior 0.64 m by 0.64 m; support top 0.005 m; roof underside 0.30 m; side/back/roof thickness 0.010 m; initial 8 cylinders, each radius 0.024 m and height 0.075 m. These are benchmark proposals and must be checked against actual assets before freezing them.

```
[carry3d]
mode = "ATTACHED_SUPPORT_CONTACT_PLANAR"
object_center_z_m = 0.0425
max_z_deviation_m = 0.001
max_upright_tilt_rad = 0.035
allow_yaw = true
allow_vertical_overflight = false
support_contact_during_carry = true

[robot3d]
asset_revision = "UNPINNED_BLOCKING_G7"
gripper_asset_revision = "UNPINNED_BLOCKING_G7"
tcp_site_name = "UNSET_BLOCKING_G7"
tool_to_object_transform_source = "UNSET_BLOCKING_G7"
controller_or_replay = "UNSET_BLOCKING_G7"
```

Reject unresolved blocking keys. Pin contact exclusions and interval verification, test cupboard self/fixed collisions and height/tilt between nodes, and distinguish rigid attachment assistance from dynamic grasp success. The arm links are not constrained to the support plane.

G8 - complete repair and achieved-state target evaluation

Implement sparse 3D counterparts of R0-R6 before dense cases. Search real relocation orders and motions; the new target planner starts at the achieved end state and uses no frozen-start connector. Record fresh observations after each physical/sim move for validation. Primary proposals remain fixed; abort if deviations invalidate execution assumptions.

Use top, front and side views plus object height/tilt traces. Preserve the move-once history and distinguish kinematic replay, dynamic/controller simulation and hardware. A failed finite IK/motion search remains unknown. Do not invent five necessary blockers to match a 2D density target.

G9 - separate 3D data, training and evaluation

Reuse `data.py`, `model.py`, `learn.py` and `evaluate.py` with a suite configuration. Run 20 audited roots before a separate 300-root pilot and conditional expansion. Train 3D separately initially; 2D-to-3D weight transfer is an additional experiment. Repeat B0-B4 and claim-specific ablations with the same no-target-feedback policy.

No 3D result may inherit a 2D feasibility certificate or an assumed real-world controller outcome. Robot/simulator asset hashes and contact/assistance flags must accompany all reported witnesses.

10. G10 and staged real-robot evaluation

Scripted long-horizon gate

Run at least three original target actions with actual achieved scene continuity. For each next action, capture a fresh initial observation and obtain one new complete repair proposal. Within that repair episode, keep object selections/destinations fixed and preserve moved history. After completion, plan and execute the target. Reset history only after successful completion or explicit abandonment of that target episode.

Construct a case where a locally useful repair interferes with a later goal. Show a timeline of initial observations, committed repairs, achieved states, target evaluation and final script outcome. The model has no plan-suffix input in this version, so this gate measures a limitation rather than claiming globally optimal long-horizon planning.

Report action success and complete-script success separately. Do not restore simulator snapshots between successive task actions. Any practical reattempt/replanning policy is a separately named variant and its extra observations, proposals and target calls are counted.

Real-robot prerequisites

The documents do not authorize live actuation. First pin the real robot/gripper/tool profile, camera calibration, object tracking, pose/shape provenance, valid workspace and controller limits. Test static scene registration and attachment/grasp conventions. Establish measured placement error and a justified clearance margin; the 5 mm simulation value is not automatically a hardware tolerance.

Run offline kinematic checks, then declared dynamic/controller simulation if available, and finally explicitly authorized hardware trials. Track force/contact, slip and uprightness for support-contact transport as needed by the actual apparatus. A simulation weld cannot replace physical validation of the grasp.

The observed repair execution may deviate from the nominal witness. Reobserve to validate the next committed motion; permit low-level planning to that same destination, counted separately. If required pose/grasp/identity assumptions fail, abort and record the failure. Do not silently issue a different high-level repair and count it as one-shot success.

Real trial record

Initial calibrated scene, model version, input provenance, committed assignment/order, planned and achieved endpoint poses, gripper and robot states, low-level retries, target-planning budget/result, video, intervention time, success criteria and abort reason. Report `HARDWARE_SUCCESS` only after the complete target goal is physically achieved and observed. Keep kinematic, simulation and hardware result columns distinct.

The first physical cases should be sparse, inspectable and repeatable. This is a staged engineering plan, not a prediction that simulation training will transfer without adaptation.

11. Visualizations, reporting and reproducibility

Every stage needs actual scene-backed visual evidence. Use consistent axes/camera, explicit meter units, physical cell IDs, full arm and gripper, target goal, chosen assignment, moved history and evidence scope. Avoid clipped fingers or missing obstacles. Use color plus text for positive, negative and unresolved statuses.

Draw a predicted endpoint arrow as a proposal, not an executed motion. Render paths and GIF frames from stored trajectories only. For rejection, show the certified first offending pair/interval where available; for timeout, show `UNRESOLVED` and the exhausted budget. Never fabricate a smooth motion between endpoints to stand in for a missing path.

Primary diagnostic plots may display reference envelopes only in clearly marked offline comparison panels. The network's clean input images must be saved separately and must not contain those overlays, target planner paths or success annotations.

Stage report template

1. Stage, document versions, exact protocol and objective.
2. Repository/profile/config/split hashes and input instructions read.
3. Changed files, responsibilities and approximate logical line counts.
4. Per-case static, relocation, target, simulation and hardware statuses.
5. Complete witnesses, certificate scopes, unknowns and unavailable cases.
6. Actual commands, test/replay outcomes and runtime/call budgets.
7. Paths to scene PNGs, GIFs, event traces and machine-readable evidence.
8. Primary-input leakage audit and target-feedback audit.
9. Deviations from the guide, their justification and limitations.
10. Explicit next-stage readiness and stop before the next gate.

Immutable evidence and versioning

Keep archived G0-G3 results byte-unchanged. An updated source function may still replay them using a compatibility adapter, but record the adapter and do not reinterpret old labels. New primary query records use a new schema/protocol version. Teacher generation changes require fresh labels and provenance; checkpoint reload must reject an incompatible schema or objective.

Save resolved configs, library versions, runtime/device details, complete deterministic seeds and asset hashes in the manifest. Distinguish deterministic node/evaluation caps from wall-time caps; wall-time truncation may change outcomes. Preserve failed attempts and their first reasons.

For broad evaluation, use independent-root confidence intervals and keep root/query/pool counts separate. Neither one dense G3 root nor many siblings of it constitute a new benchmark. Hardware trials are additional evidence, not replacements for the controlled simulation ablation.

12. Worked logic checks and release checklist

Example A - original route hits four objects

A stored reference collides with A, B, C and D. In a cloned same-state evaluation, another route completes the target without moving anything. The primary action-level label for all-stay is positive with minimum zero. The old frozen-route blocker set remains valid diagnostic evidence about the reference only. Do not label the four objects as necessary repair targets.

Example B - one required target slot is occupied

Target goal is one fixed pose. Object A occupies that pose and cannot move unless selected. The target and A geometrically overlap at the required final goal. This is a sound action-level obstruction under the declared scope. If a verified relocation of A followed by a complete target motion succeeds, minimum one is certified. If the target can use another slot/orientation, recheck the premise; the certificate may no longer apply.

Example C - two candidate destinations and unknown execution

Assignments A-to-P/B-to-Q and A-to-Q/B-to-P may both be static-valid. Give a complete-assignment positive only to each independently witnessed repair-plus-target sequence. If the second search times out, it remains unknown. Both-to-P is static-invalid. Do not treat the unknown assignment as a negative in BCE or as a negative competitor in the joint softmax.

Example D - an executable repair still fails the target evaluation

All predicted relocations execute, but the final target planner exhausts its budget. Count the episode unsuccessful within its operational budget; record complete-assignment truth as unresolved unless another sound proof/witness exists. Do not try the next proposal and retain a one-shot success label. Such retries belong in a separate variant.

Release checklist

Check	Gate
Legacy frozen evidence unchanged and correctly scoped	G3.5
No reference required; action/goal interface explicit	G3.5
R0-R6 report with genuine full-robot evidence/unknowns	G3.5
Zero target feedback before commit or between moves	G3.5 and every evaluation
Initial proposal fixed, actual execution state continuous	G3.5 and every execution
Informative positive/negative labels with unknown masking	G4
Full-energy Gibbs and whole-assignment head tested	G4/G5
Matched baselines, valid denominators, held-out roots	G6/G9
Pinned 3D assets, contact and height/tilt verification	G7/G8
Separate dynamic and hardware evidence	G8 onward
Long-horizon continuity without suffix claims	G10

The key change from v1.6 is a new problem contract, not the removal of one input tensor. Preserve working geometry and execution code; change labels, model semantics, target-call boundaries and claims together. The companion specification defines the mathematics; this guide defines implementation order and acceptance.